School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

Fall Semester 2026-27

ASSESSMENT X – PROJECT DEMO AND OUTCOME

DagOpt: DAG-Based Optimization of Basic Blocks

Common Subexpression and Redundant Computation Elimination using Directed Acyclic Graphs

Programme & Branch	B.Tech (CSE & Specialization) & SCOPE
Course Name & Code	Compiler Design Lab (BCSE307P)
Class Number	VL2026270103361
Slot	L15+L16
Faculty Name	Dr. Bhawana Tyagi

Team Members

S.No	Name	Register Number
1	Harshit Pandey	24BCT0173
2	Ankush Kumar	24BCT0254
3	Lakshya Kulshrestha	24BCT0299

Abstract

Intermediate code produced by a compiler front end often computes the same value several times. DagOpt is a command-line tool that removes this redundancy inside basic blocks. It reads three-address code, splits it into basic blocks, builds a directed acyclic graph (DAG) for each block in which every distinct value is a single node, and regenerates optimized code from the graph. Common subexpressions (including those written with swapped operands), constant expressions and algebraic identities are removed, and unused temporaries are dropped using liveness information. Every optimized block is checked against the original by running both on random inputs. On ten test programs the tool reduces the arithmetic operations from 34 to 15 and every result passes the equivalence check; the 36 unit tests all pass.

1. Introduction

1.1 Problem Statement

Compilers translate source programs into intermediate code, usually three-address code (TAC). Code produced directly from the front end is rarely efficient: the same expression is often computed several times, and values are recomputed even when nothing they depend on has changed. These redundant computations waste execution time and, at the machine level, registers and memory traffic. Our assigned topic is to **implement a DAG-based optimization technique for basic blocks, construct the DAG representation and use it to eliminate common subexpressions and redundant computations.**

1.2 Why a DAG?

A basic block is a straight-line sequence of instructions with one entry and one exit. In its DAG, each distinct value is a single node and every variable that holds the value is a label on the node. When an instruction would compute a value that already has a node, the node is reused, which is exactly the elimination of a common subexpression. Redundancy is found in a single pass over the block, without a separate data-flow analysis.

1.3 Objectives and Completion Status

No.	Objective	Where it is implemented	Status
1	Design a TAC input format and a parser	tac.py: six instruction forms read with regular expressions, syntax errors reported with the line number	Completed
2	Partition the TAC into basic blocks	blocks.py: leader rules, block terminators, control flow edges	Completed
3	Build a DAG per block with value sharing	dag.py: hash table of (operator, left, right), variable labels on nodes	Completed
4	Eliminate common subexpressions (including a + b and b + a) and redundant computations	dag.py, simplify.py, liveness.py: commutative keys, constant folding, algebraic identities, dead node removal	Completed

No.	Objective	Where it is implemented	Status
5	Produce optimized code from the DAG and compare it with the original	codegen.py, metrics.py: code regeneration, before/after counts, JSON and CSV export	Completed
6	Validate that the optimization preserves meaning	interp.py, verify.py and 36 unit tests	Completed

2. Literature Survey

Code optimization on basic blocks has a long history. The works below shaped our approach.

No.	Work	Contribution	Relevance to this project
1	Aho, Lam, Sethi & Ullman, Compilers: Principles, Techniques, and Tools (2nd ed., 2006), Ch. 8–9	Defines basic blocks, leaders and flow graphs, the DAG representation of basic blocks, and the DAG construction algorithm	Primary reference for our block splitting and DAG algorithm
2	Cocke & Schwartz, Programming Languages and Their Compilers (1970)	Early work on common subexpression elimination, value numbering and local optimization	Origin of the idea of assigning one identity to each computed value
3	Aho & Ullman, "Optimization of Straight Line Code", SIAM J. Computing, 1972	Theory of optimizing straight-line code using DAGs	Justifies DAGs as the right structure for a basic block
4	Aho, Johnson & Ullman, "Code Generation for Expressions with Common Subexpressions", JACM, 1977	Shows how to generate code from DAGs that contain shared nodes	Guided turning the optimized DAG back into code
5	Briggs, Cooper & Simpson, "Value Numbering", Software: Practice & Experience, 1997	Practical hash-based value numbering and its extension beyond a single block	Hash-table lookup of (operator, operands) is the same idea as our DAG node sharing
6	Cooper & Torczon, Engineering a Compiler (2nd ed., 2011)	Local value numbering, algebraic identities and redundancy elimination	Source of the algebraic and commutativity rules we use
7	Muchnick, Advanced Compiler Design and Implementation (1997)	Survey of machine-independent optimizations, including CSE and constant folding	Background for the optimizations the tool supports
8	Lattner & Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation", CGO, 2004	Modern compiler infrastructure with passes such as early CSE and instruction combining	Shows how block-level CSE is used in a production compiler

2.1 Gap and Our Contribution

DAG-based local optimization and local value numbering are two views of the same technique. Textbooks describe the algorithm but rarely give a runnable tool that shows the DAG and the effect of optimization, and production compilers hide these steps inside large frameworks. DagOpt is a small, self-contained tool that keeps each stage separate, shows the DAG of every

block, reports what every optimization removed, and checks that the optimized code behaves like the original.

3. Design and Implementation

The tool is a Python 3.10+ package (dagopt) in which each stage is a separate module that takes the output of the previous stage. This allows every stage to be tested on its own.

Stage	Module	Task
1. Parsing	tac.py	Read TAC into instruction objects; C-style division and modulo (truncation toward zero)
2. Block splitting	blocks.py	Leaders: first instruction, every label, instruction after a jump; control flow edges
3. DAG construction	dag.py, simplify.py	Value sharing, commutative keys, constant folding, algebraic identities
4. Liveness	liveness.py	Variables needed after a block, including those read by its final branch
5. Code generation	codegen.py	Regenerate TAC from the live nodes
6. Verification	interp.py, verify.py	Run original and optimized block on random inputs and compare
7. Output and reports	dotexport.py, metrics.py, main.py, tools/render_dag.py	Graphviz and PNG output of the DAGs, statistics table, JSON and CSV, command-line driver

3.1 DAG Construction

Instructions are processed in order. A table maps each variable to the node that holds its value, and a hash table maps a key (operator, left node, right node) to an existing node. A hit is a common subexpression and the node is reused. For + and * the two operand nodes are put in a fixed order, so a + b and b + a share a node. Keys use node identities and not variable names, so a reassigned variable automatically gets a different node and the sharing stays safe.

```
for each instruction x = y OP z:
    l = node_for(y); r = node_for(z)
    result = try_simplify(OP, l, r)
    if result is a constant fold and folding is enabled:
        bind x to the constant node
    elif result is an algebraic identity and algebra is enabled:
        bind x to the surviving node or constant
    else:
        key = (OP, l, r)   # operands ordered for + and *
        if key in table: reuse node (CSE hit) else create node
        bind x to the node
```

Constants are folded for negation, +, -, *, / and %. The identities x + 0, 0 + x, x * 1, 1 * x, x * 0, 0 * x, x - 0, x - x and x / 1 are applied. The identity x / x = 1 is deliberately not used, because it is wrong for x = 0. Each result is tagged as folding or algebra, so each can be switched off (--no-fold, --no-algebra, --no-commute) and each has its own counter.

3.2 Liveness and Dead Nodes

The live-out set of a block contains every program variable that appears in it (temporaries such as t1 or _n1 are excluded) and every variable read by the final branch. All nodes reachable from the nodes of these variables are live; the other nodes are dead and are not emitted. This removes unused temporaries and copies, and it guarantees that a loop condition is never removed.

3.3 Code Generation

1. **Naming.** Leaves keep their variable or constant. An operation node is named by one of its live labels, preferring a program variable over a temporary; otherwise a fresh temporary _n1, _n2, ... is created.
2. **Emission.** One instruction is emitted per live operation node, and every other live label of the node becomes a copy (for example y = x). A repeated computation therefore becomes a cheap copy.
3. **Scheduling.** A value must be computed before it is used, and the original value of a variable must be read before the variable is overwritten. Ready instructions are emitted in node creation order. An instruction such as i = i + 1 that reads and writes the same variable needs no special treatment.
4. **Cycle breaking and control flow.** If the dependencies form a real cycle, as in a swap, the original value of a variable is first copied to a fresh temporary. Labels at the start of a block are written first, and the operands of the final conditional branch are rewritten to the chosen node names.

3.4 Verification

An interpreter executes the original and the optimized block from the same random initial values, with C-style division and modulo, and compares all live-out variables over a chosen number of trials (--verify N, --seed S for repeatability). A random block generator is used to fuzz the optimizer.

3.5 Worked Example

Input TAC	Optimized TAC

`t1 = a + b`
`t2 = a + b`
`t3 = b + a`
`x = t1 * c`
`y = t2 * c`
`z = t3 + x`
	`t1 = a + b`
	`x = t1 * c`
	`y = x`
	`z = t1 + x`

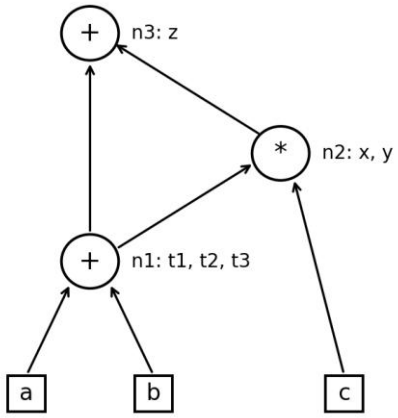


Figure 1: DAG of the worked example (circles are operations, boxes are leaves)

Node n1 (+) carries the labels t1, t2 and t3, node n2 (*) carries x and y, and n3 (+) carries z. The six arithmetic operations of the input need only three nodes, and the copies for the unused temporaries t2 and t3 are removed, so the block shrinks from 6 to 4 instructions.

4. Demonstration

The demonstration shows the complete pipeline on real input. Each step is a single command.

Step	Command	What it shows
1	python -m pytest -v	All 36 tests pass
2	python -m dagopt.main tests/cases/T01.tac --metrics --verify 500 --seed 1	Optimized TAC, the metrics table and Verify = True
3	python -m dagopt.main <file>.tac --dot-dir dots, then python tools/render_dag.py	The DAG of each block as a picture; shared nodes are shaded and dead nodes are dashed
4	python -m dagopt.main <constants file>.tac --metrics with and without --no-fold or --no-algebra	The effect of each optimization on the same program
5	The loop sample	The label and the branch are preserved and the repeated x + y inside the loop is removed
6	The swap sample	Cycle breaking with a temporary, verified correct

5. Results and Analysis

All results were produced by the tool with --metrics --verify 500 --seed 1. Instruction counts do not include labels.

Program	Instr. before	Instr. after	Arith. before	Arith. after	CSE hits	Folded	Algeb.	Verif.
Worked example	6	4	6	3	3	0	0	True
CSE sample	3	3	3	2	1	0	0	True

Program	Instr. before	Instr. after	Arith. before	Arith. after	CSE hits	Folded	Algeb.	Verif.
Constants sample	6	6	4	0	0	1	3	True
Loop sample	6	6	4	3	1	0	0	True
Swap sample	3	4	0	0	0	0	0	True
T01	4	4	4	3	1	0	0	True
T02	3	2	3	2	1	0	0	True
T05	6	6	4	0	0	4	0	True
T06	6	6	4	0	0	4	0	True
T07	3	3	2	2	0	0	0	True
Total	**46**	**44**	**34**	**15**	**7**	**9**	**3**	**All True**

5.1 Contribution of Each Technique

Technique	Operations removed	Share of the 19 removed
Common subexpression sharing (including commutative)	7	37%
Constant folding	9	47%
Algebraic identities	3	16%
Total	**19 of 34 (56%)**	100%

5.2 Discussion

- Across the ten programs the arithmetic operations fall from 34 to 15, a reduction of 56%, and every optimized program is equivalent to its original on 500 random inputs per block.
- The instruction count falls only from 46 to 44, because a removed computation is often replaced by a copy. The arithmetic count is the better measure, since a copy is much cheaper than a multiplication or a division.
- Constant folding gives the largest saving on the programs with constant expressions (T05, T06), and common subexpression sharing gives the saving on the programs with repeated expressions.
- The swap sample is one instruction longer than its input. The result is correct, but a smaller output is possible.
- The 36 unit tests cover the parser (12), code generation (5), the interpreter (4), block splitting (3), the DAG (3), simplification (3), verification (3), DOT export (1), the end-to-end run (1) and metrics (1).

6. Individual Contribution

Member	Work done	Modules and tests
Harshit Pandey (24BCT0173)	Problem identification, objectives and literature survey; TAC parser with syntax errors and C-style division and modulo; constant folding and algebraic identities; liveness and dead node removal; test programs T01 to T07 and the end-to-end test	tac.py, simplify.py, liveness.py; test_tac, test_simplify, test_e2e
Ankush Kumar (24BCT0254)	Basic block splitting with leaders, terminators and control flow edges; TAC interpreter; random-input equivalence verification and the random block generator; DOT export of the DAGs; metrics tables with JSON and CSV export	blocks.py, interp.py, verify.py, dotexport.py, metrics.py; test_blocks, test_interp, test_verify, test_dotexport, test_metrics
Lakshya Kulshrestha (24BCT0299)	DAG construction with value sharing and commutative keys; code generation from the DAG with scheduling and cycle breaking; the command-line tool; the repository and the live demo	dag.py, codegen.py, main.py, cli.py; test_dag, test_codegen

7. Outcome

The outcome of the project is a manuscript describing the tool and its evaluation, titled **"DagOpt: A Verified and Inspectable DAG-Based Optimizer for Basic Blocks of Three-Address Code"**. The manuscript is included in Appendix A. It presents the design, the tagged simplification that allows each optimization to be measured separately, the dependency-aware code generation, the verification method and the results of Section 5.

8. Limitations and Future Work

- Liveness is local to a block: all program variables of a block are assumed live at its end, so an assignment that is dead because of later blocks is kept. A data-flow analysis over the control flow graph would remove it.
- Verification is by random testing of each block and is not a formal proof.
- Strength reduction (for example x * 2 as x + x) and double negation are not implemented.
- Cycle breaking can use one more instruction than necessary, as in the swap sample.

9. Conclusion

DagOpt implements the complete path from three-address code to verified optimized code using DAGs of basic blocks. All six objectives are met: the tool removes common subexpressions, constant expressions and algebraic redundancies, drops unused temporaries, regenerates correct code with labels and branches preserved, and checks every result against the original. On the test programs it removes 56% of the arithmetic operations, and the separate switches and counters make it easy to see which technique is responsible for each saving.

References

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson/Addison-Wesley, 2006.
2. J. Cocke and J. T. Schwartz, Programming Languages and Their Compilers: Preliminary Notes. Courant Institute, New York University, 1970.
3. A. V. Aho and J. D. Ullman, "Optimization of straight line code," SIAM Journal on Computing, vol. 1, no. 1, pp. 1–19, 1972.
4. A. V. Aho, S. C. Johnson and J. D. Ullman, "Code generation for expressions with common subexpressions," Journal of the ACM, vol. 24, no. 1, pp. 146–160, 1977.
5. P. Briggs, K. D. Cooper and L. T. Simpson, "Value numbering," Software: Practice and Experience, vol. 27, no. 6, pp. 701–724, 1997.
6. K. D. Cooper and L. Torczon, Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011.
7. S. S. Muchnick, Advanced Compiler Design and Implementation. Morgan Kaufmann, 1997.
8. C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in Proc. IEEE/ACM CGO, 2004, pp. 75–86.

Appendix A: Manuscript

DagOpt: A Verified and Inspectable DAG-Based Optimizer for Basic Blocks of Three-Address Code

Harshit Pandey, Ankush Kumar, Lakshya Kulshrestha

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India

Abstract— Local optimization of basic blocks with directed acyclic graphs (DAGs) is a classical compiler technique, but it is usually described only as an algorithm. We present DagOpt, a small command-line tool that builds a DAG for every basic block of three-address code, eliminates common subexpressions, constant expressions and algebraic redundancies, and regenerates optimized code. Folding, algebraic identities and commutative sharing are separately switchable and separately counted, code generation schedules instructions so that no value is overwritten before it is read, and each optimized block is checked against the original on random inputs. On ten test programs the tool removes 19 of 34 arithmetic operations and all outputs pass the check.

Keywords— basic blocks, directed acyclic graph, common subexpression elimination, value numbering, code optimization, three-address code.

I. Introduction

Code generated by a compiler front end often recomputes values that are already available. Representing a basic block as a DAG gives each distinct value a single node, so a repeated computation is found by a table lookup [1], [3]. The same idea is known as local value numbering [2], [5]. Descriptions of the method seldom come with a tool that shows the graph and measures the result. Our contribution is such a tool, together with an automatic equivalence check.

II. Method

The input is three-address code with copies, negation, binary operations, labels, jumps and conditional jumps. The code is split into basic blocks at leaders. For each block, a DAG is built by processing the instructions in order. A hash table keyed on (operator, left node, right node) returns an existing node for a repeated computation, and the operands of + and * are ordered so that a + b and b + a share a node. Constant folding and algebraic identities are applied before the lookup and are tagged, so each can be disabled and counted on its own. The identity x / x = 1 is omitted because it fails for x = 0, and division and modulo truncate toward zero as in C.

A node is live if it is reachable from a variable that is needed after the block or read by the block's final branch. Only live nodes are emitted. Each operation node produces one instruction named after a live label, and the other labels of the node become copies. The instructions are ordered so that values are computed before use and original values are read before they are overwritten; a real dependency cycle, such as a swap, is broken with a fresh temporary.

III. Evaluation

Each optimized block was compared with the original by an interpreter on 500 random inputs. On ten programs the number of arithmetic operations fell from 34 to 15 (56%): 7 operations were removed by common subexpression sharing, 9 by constant folding and 3 by algebraic identities. The instruction count fell from 46 to 44 because removed computations are often replaced by copies. All outputs passed the equivalence check, and 36 unit tests pass.

IV. Limitations and Conclusion

The liveness analysis is local to a block and the verification is by testing and not by proof. Future work is data-flow liveness over the whole control flow graph, strength reduction, and evaluation on larger programs. DagOpt shows that the classical DAG method can be implemented as a compact, testable tool whose effect is easy to inspect.

References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., 2006.

[2] J. Cocke and J. T. Schwartz, Programming Languages and Their Compilers, 1970.

[3] A. V. Aho and J. D. Ullman, "Optimization of straight line code," SIAM J. Comput., vol. 1, no. 1, pp. 1–19, 1972.

[4] A. V. Aho, S. C. Johnson and J. D. Ullman, "Code generation for expressions with common subexpressions," J. ACM, vol. 24, no. 1, pp. 146–160, 1977.

[5] P. Briggs, K. D. Cooper and L. T. Simpson, "Value numbering," Softw. Pract. Exper., vol. 27, no. 6, pp. 701–724, 1997.